

Security Assessment Report

Vulnerable Web Application Security Lab

Assessment Date	2026-06-07
	C:\Users\moeez\vuln-app-cc
Version / Commit	0.1.0 / backend 1.0.0 / 56c17b9
	Codex Security Audit Report Skill
Final Rating	High Risk (20/100)

Confidential — intended for authorized stakeholders only.

Table of Contents

Table of Contents	2
Executive Summary	6
Key Strengths	6
Key Risks	6
Security Score	6
Overall score: 20/100 — High Risk	6
Scope and Methodology	6
Standards Used	7
Limitations	7
Application Overview and Attack Surface	7
OWASP ASVS Assessment	7
Findings	8
F-001: Unauthenticated Database Download Exposes User Records	8
Description	8
Technical Details	8
Evidence	8
Impact	8
Exploitation Scenario	8
Remediation	9
Secure Code Example	9
F-002: Reflected Cross-Site Scripting in Search Response	9
Description	9
Technical Details	9
Evidence	9

Impact	9
Exploitation Scenario	9
Remediation	9
Secure Code Example	10
F-003: Stored Cross-Site Scripting Through Username Rendering	10
Description	10
Technical Details	10
Evidence	10
Impact	10
Exploitation Scenario	10
Remediation	10
Secure Code Example	10
F-004: Hardcoded Session Signing Secret	11
Description	11
Technical Details	11
Evidence	11
Impact	11
Exploitation Scenario	11
Remediation	11
Secure Code Example	11
F-005: No Rate Limiting or Account Lockout on Authentication	11
Description	12
Technical Details	12
Evidence	12
Impact	12
Exploitation Scenario	12
Remediation	12

Secure Code Example	12
F-006: Missing CSRF Protection on State-Changing Requests	12
Description	12
Technical Details	12
Evidence	13
Impact	13
Exploitation Scenario	13
Remediation	13
Secure Code Example	13
F-007: Security Headers Are Not Configured	13
Description	13
Technical Details	13
Evidence	13
Impact	13
Exploitation Scenario	13
Remediation	14
Secure Code Example	14
F-008: Unpinned and Inconsistent Dependency Manifests	14
Description	14
Technical Details	14
Evidence	14
Impact	14
Exploitation Scenario	14
Remediation	14
Secure Code Example	15
F-009: Unresolved Merge Conflict Markers in Dashboard Template	15

Description	15
Technical Details	15
Evidence	15
Impact	15
Exploitation Scenario	15
Remediation	15
Secure Code Example	15
F-010: No Security Test, CI, or Vulnerability Response Evidence	16
Description	16
Technical Details	16
Evidence	16
Impact	16
Exploitation Scenario	16
Remediation	16
Secure Code Example	16
Risk Matrix	16
Remediation Roadmap	17
NIST SSDF Recommendations	17
Final Security Rating	17
High Risk	17
Appendix: Standards Mapping and Evidence Notes	17

Executive Summary

The application is intentionally insecure for training, but the current repository state still contains high-impact flaws that would be unacceptable outside an isolated lab. The most serious confirmed issue is an unauthenticated database download route that exposes the SQLite database file. Cross-site scripting remains present through unescaped user-controlled HTML output, and the session signing secret is hardcoded in source. CSRF protections, rate limiting, security headers, dependency pinning, and SDLC evidence are also incomplete. Some documentation is stale relative to the current implementation: the source now uses parameterized SQL for signup/login and bcrypt password hashing, while README/TDD/AGENTS still describe SQL injection and MD5 as active flaws.

Critical	High	Medium	Low	Informational
0	4	5	1	0

Key Strengths

- Current signup and login SQL queries use parameter placeholders rather than string concatenation.
- Current password hashing uses bcrypt with generated salts and a cost factor of 12.
- The project clearly labels itself as an educational vulnerable application and warns against production deployment.
- The codebase is small enough for students to trace data flow from request handling to storage and rendering.

Key Risks

- Unauthenticated database download allows direct exposure of user data and password hashes.
- Stored and reflected XSS can execute attacker-controlled script in users' browsers.
- Hardcoded session secret allows session forgery if source or image is shared.
- No CSRF protection or rate limiting exists for authentication/session flows.
- Documentation and UI advertise vulnerabilities that no longer match current source, increasing training and assessment confusion.

Security Score

Overall score: 20/100 — High Risk

Started at 100 and subtracted for confirmed current findings: 4 High findings (48), 5 Medium findings (30), and 1 Low finding (2), yielding 20/100. SQL injection and MD5 were not counted because the current code uses parameterized queries and bcrypt. The intentional lab context explains the risk but does not reduce the technical exposure if run on a reachable network.

Scope and Methodology

Static application security assessment of the local repository C:\Users\moeez\vuln-app-cc, including FastAPI backend source, HTML/CSS/JavaScript frontend templates, SQLite database artifact presence, dependency manifests, and product/technical documentation. The application is intentionally vulnerable for education; findings distinguish current confirmed implementation risk from documented lab intent.

Manual code review and repository inventory using safe static analysis only. Reviewed authentication, session management, routes, database access, output rendering, dependency declarations, documentation, and evidence of SDLC controls. No destructive testing, third-party testing, or live exploitation was performed.

Standards Used

- OWASP ASVS 5.0.0
- OWASP Top 10:2025
- MITRE CWE
- CVSS v3.1
- NIST SSDF SP 800-218 v1.1

Limitations

The assessment did not include authenticated dynamic testing, dependency vulnerability lookups against live advisory databases, browser-based DAST, or review of infrastructure not present in the repository. SQL injection and MD5 password storage described in older documentation were not treated as confirmed current code findings because current source uses parameterized login/signup queries and bcrypt.

Application Overview and Attack Surface

FastAPI web application with SQLite persistence, Starlette signed-cookie SessionMiddleware, vanilla HTML/CSS/JavaScript templates, and form-based signup/login/logout/search flows. Static assets are served by FastAPI. The users table stores id, username, email, and password hash. The dashboard is protected only by presence of user_id in the session cookie.

- GET /signup and POST /signup registration form
- GET /login and POST /login authentication form
- GET /welcome authenticated dashboard rendering username into HTML
- GET /search reflected HTML response using q plus database result fields
- GET /download/db unauthenticated SQLite database download
- GET /logout state-changing session clear action
- Static assets under /static/css and /static/images
- Repository-shipped vulnerable_app.db database artifact

OWASP ASVS Assessment

ASVS Area	Status	Evidence	Related Findings
V1 Architecture, Design and Threat Modeling	Partially Met	PRD/TDD describe the educational threat model, but there is no deploy-time guardrail preventing non-lab exposure and current docs are stale relative to code.	F-009, F-010
V2 Authentication	Partially Met	Login checks bcrypt hashes, but no throttling, lockout, or automated abuse controls are implemented.	F-005
V3 Session Management	Not Met	SessionMiddleware uses a source-controlled hardcoded secret and dashboard authorization trusts cookie session state only.	F-004
V4 Access Control	Not Met	The database download route has no authentication or authorization check.	F-001

ASVS Area	Status	Evidence	Related Findings
V5 Validation, Sanitization and Encoding	Not Met	Search query, database result fields, and username replacement are inserted into HTML without output encoding.	F-002, F-003
V8 Data Protection	Partially Met	Passwords are bcrypt-hashed in current code, but the SQLite database file is present at project root and directly downloadable.	F-001
V9 Communications	Not Assessed	TLS termination and deployment configuration are not present in the repository.	—
V11 Business Logic	Partially Met	Core flows are simple and explicit, but logout is implemented as a GET state-changing request and no anti-automation controls exist.	F-006, F-005
V12 Files and Resources	Not Met	A sensitive database file is served as a downloadable resource without access control.	F-001
V14 Configuration	Not Met	Secrets are hardcoded and security headers/dependency pinning/CI evidence are absent.	F-004, F-007, F-008

Findings

F-001: Unauthenticated Database Download Exposes User Records

Severity	High	CVSS	8.6 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:HI/I:L/A:L
	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor; CWE-862: Missing Authorization		OWASP Top 10:2025 A01 Broken Access Control; A02 Cryptographic Failures
ASVS	ASVS V4 Access Control; V8 Data Protection; V12 File and Resources	Location	backend/app/api/routes/auth.py:50-54; vulnerable_app.db at repository root

Description

The `/download/db` endpoint returns the SQLite database file without checking authentication or authorization.

Technical Details

`download_db` constructs the project-root database path and returns `FileResponse` directly. The route is registered without dependency checks, session validation, role checks, or environment gating. The database artifact also exists in the repository root.

Evidence

```
@router.get('/download/db') def download_db(): db_path = os.path.join(_PROJECT_ROOT, 'vulnerable_app.db');
return FileResponse(path=db_path, filename='vulnerable_app.db'). Repository inventory confirmed
vulnerable_app.db is present.
```

Impact

Any network user can obtain all stored usernames, emails, and password hashes. If the database contains real credentials or reused passwords, this can lead to account takeover and privacy exposure.

Exploitation Scenario

An unauthenticated visitor requests the database export endpoint and receives the full SQLite file, then inspects local tables offline.

Remediation

Remove the endpoint from non-training builds. If a download feature is required, require authentication, explicit authorization, audit logging, and export only least-privilege data. Keep database files outside web-accessible paths and exclude them from source control.

Secure Code Example

```
from fastapi import Depends, HTTPException

def require_admin(request: Request):
    if not request.session.get('is_admin'):
        raise HTTPException(status_code=403)

@router.get('/download/db')
def download_db(_: None = Depends(require_admin)):
    raise HTTPException(status_code=404) # disabled outside isolated lab builds
```

F-002: Reflected Cross-Site Scripting in Search Response

Severity	High	CVSS	7.1 CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/L:A:L
	CWE-79: Improper Neutralization of Input During Web Page Generation		OWASP Top 10:2025 A03 Injection
ASVS	ASVS V5 Validation, Sanitization and Encoding	Location	backend/app/api/routes/auth.py:57-80

Description

The search endpoint directly interpolates the q query parameter into an HTML response without escaping.

Technical Details

Although the SQL query uses parameters, the response body is assembled with f-strings. The raw query string and result fields are concatenated into HTML and returned as HTMLResponse.

Evidence

```
html = f'<h2>Search Results for: {q}</h2>'; html += f'<li>{row[0]} ({row[1]})</li>'; return HTMLResponse(html).
```

Impact

A crafted link can execute script in a victim's browser in the application origin, enabling session actions, content manipulation, or data access available to the victim.

Exploitation Scenario

An attacker sends a user a search URL containing HTML/JavaScript-like content; when opened, the application reflects it into the page as executable markup.

Remediation

Render search responses through templates with automatic escaping or explicitly HTML-escape all user-controlled values before insertion. Prefer returning JSON to client-rendered textContent where appropriate.

Secure Code Example

```
from html import escape

safe_q = escape(q or '')
html = f'<h2>Search Results for: {safe_q}</h2>'
for row in results:
    html += f'<li>{escape(row[0])} ({escape(row[1]})</li>'
return HTMLResponse(html)
```

F-003: Stored Cross-Site Scripting Through Username Rendering

Severity	High	CVSS	7.1 CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:L
	CWE-79: Improper Neutralization of Input During Web Page Generation		OWASP Top 10:2025 A03 Injection
ASVS	ASVS V5 Validation, Sanitization and Encoding	Location	backend/app/services/auth_service.py:18-36; backend/app/api/routes/auth.py:89-93; frontend/templates/dashboard.html:63-66

Description

Username are accepted without server-side validation and later inserted into the dashboard HTML by string replacement without output encoding.

Technical Details

signup stores username directly in SQLite. welcome_page reads dashboard.html and replaces {{username}} with the session username string. Because the dashboard template uses HTML context, markup in username can become executable.

Evidence

conn.execute(query, (username, email, hashed)); username = request.session.get('username', 'Unknown'); html = html.replace('{{username}}', username); dashboard contains {{username}}.

Impact

A registered user can persist malicious markup in their username and have it execute when the dashboard renders that value, potentially affecting themselves or any future admin/user views that reuse the same field.

Exploitation Scenario

An attacker registers with a username containing active markup. After authentication, the dashboard renders that stored value in the page origin.

Remediation

Use a templating engine with autoescaping, validate username character sets and length on the server, and encode output by context. Do not perform raw string replacement into HTML.

Secure Code Example

```
from fastapi.templating import Jinja2Templates
templates = Jinja2Templates(directory=TEMPLATES_DIR)

@router.get('/welcome')
def welcome_page(request: Request):
    if 'user_id' not in request.session:
        return RedirectResponse('/login', status_code=302)
```

```
return templates.TemplateResponse('dashboard.html', {'request': request, 'username': request.session.get('username', 'Unknown')})
```

F-004: Hardcoded Session Signing Secret

Severity	High	CVSS	8.1 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
	CWE-798: Use of Hard-coded Credentials; CWE-321: Use of Hard-coded Cryptographic Key		OWASP Top 10:2025 A02 Cryptographic Failures; A07 Identification and Authentication Failures
ASVS	ASVS V3 Session Management; V14 Configuration	Location	backend/app/main.py:27-31

Description

The Starlette SessionMiddleware secret key is a static literal committed to source code.

Technical Details

SessionMiddleware signs client-side session cookies. If the secret is known, copied across environments, or leaked with source code, an attacker can forge signed session state accepted by the application.

Evidence

```
SECRET_KEY = 'super-secret-key-12345'; app.add_middleware(SessionMiddleware, secret_key=SECRET_KEY).
```

Impact

Attackers with the secret can create or modify session cookies, potentially impersonating users or setting arbitrary authenticated session attributes.

Exploitation Scenario

A user obtains source code or a container image containing the key and uses that knowledge to craft a session cookie that includes authenticated user state.

Remediation

Load the session secret from a deployment secret manager or environment variable, require high entropy, rotate it per environment, and fail closed if missing outside lab mode. Set cookie security attributes appropriate for HTTPS deployments.

Secure Code Example

```
SECRET_KEY = os.environ.get('SESSION_SECRET_KEY')
if not SECRET_KEY or len(SECRET_KEY) < 32:
    raise RuntimeError('SESSION_SECRET_KEY must be configured with high entropy')
app.add_middleware(SessionMiddleware, secret_key=SECRET_KEY, https_only=True, same_site='lax')
```

F-005: No Rate Limiting or Account Lockout on Authentication

Severity	Medium	CVSS	6.5 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
	CWE-307: Improper Restriction of Excessive Authentication Attempts		OWASP Top 10:2025 A07 Identification and Authentication Failures
ASVS	ASVS V2 Authentication	Location	backend/app/main.py:29-33; backend/app/api/routes/auth.py:44-47

Description

The application accepts unlimited login attempts without IP, username, or session throttling.

Technical Details

No middleware, dependency, or service-level counter limits authentication attempts. Failed logins always return a normal 401 response and can be repeated quickly.

Evidence

`app = FastAPI(); app.add_middleware(SessionMiddleware, ...); app.include_router(router).` No rate-limiting middleware or lockout logic appears in the route or service.

Impact

Attackers can automate password guessing and credential stuffing, especially if the app is exposed beyond a local lab.

Exploitation Scenario

An attacker repeatedly submits candidate credentials to POST /login and observes 401 vs redirect behavior until a valid password is found.

Remediation

Add rate limiting keyed by IP and account identifier, introduce progressive delays or temporary lockouts, log suspicious activity, and add alerting for repeated failures.

Secure Code Example

```
from slowapi import Limiter
from slowapi.util import get_remote_address
limiter = Limiter(key_func=get_remote_address)

@router.post('/login')
@limiter.limit('5/minute')
def login_post(request: Request, username: str = Form(None), password: [REDACTED] = Form(None)):
    return do_login(request, username, password)
```

F-006: Missing CSRF Protection on State-Changing Requests

Severity	Medium	CVSS	6.5 CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N
	CWE-352: Cross-Site Request Forgery		OWASP Top 10:2025 A01 Broken Access Control
ASVS	ASVS V4 Access Control; V11 Business Logic	Location	frontend/templates/signup.html:56-79; frontend/templates/login.html:56-99; backend/app/api/routes/auth.py:32-47,96-100

Description

Forms and state-changing session operations do not use CSRF tokens or origin validation.

Technical Details

POST /signup and POST /login accept form submissions with no anti-CSRF token validation. Logout is performed through GET /logout, which changes session state and can be triggered cross-site.

Evidence

Signup form posts directly to /signup with no hidden token. Login fetch posts /login with credentials: same-origin but no token. logout clears request.session in a GET handler.

Impact

A victim's browser can be induced to submit unwanted state-changing requests, such as logging the user out or creating/altering session state in workflows added later.

Exploitation Scenario

A malicious page causes an authenticated browser to request /logout or submit application forms without the user's intent.

Remediation

Use signed per-session CSRF tokens for all state-changing methods, validate Origin/Referer where appropriate, and move logout to POST. Keep SameSite cookies enabled but do not rely on them as the only defense.

Secure Code Example

```
@router.post('/logout')
def logout(request: Request, csrf_token: [REDACTED] = Form(...)):
    validate_csrf(request.session, csrf_token)
    request.session.clear()
    return RedirectResponse('/login', status_code=302)
```

F-007: Security Headers Are Not Configured

Severity	Medium	CVSS	5.3 CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
	CWE-693: Protection Mechanism Failure		OWASP Top 10:2025 A05 Security Misconfiguration
ASVS	ASVS V14 Configuration	Location	backend/app/main.py:29-33

Description

The application does not configure standard browser security headers such as Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, or frame protections.

Technical Details

FastAPI is instantiated and only SessionMiddleware plus routes/static mounts are added. No middleware sets response hardening headers. This worsens XSS impact because no CSP is available as a defense-in-depth layer.

Evidence

app = FastAPI(); app.add_middleware(SessionMiddleware, secret_key=SECRET_KEY); app.include_router(router). No header middleware present.

Impact

Browser-side attacks become easier to exploit and harder to contain. XSS can execute without policy restrictions, and content sniffing/clickjacking controls are absent.

Exploitation Scenario

When a reflected or stored XSS path is triggered, the browser has no application CSP to limit inline script execution or exfiltration endpoints.

Remediation

Add response header middleware. Tune CSP for the frontend, avoid inline scripts where possible, and include X-Content-Type-Options: nosniff, Referrer-Policy, frame-ancestors/frame options, and permissions policy.

Secure Code Example

```
@app.middleware('http')
async def security_headers(request, call_next):
    response = await call_next(request)
    response.headers['Content-Security-Policy'] = "default-src 'self'; object-src 'none'; frame-ancestors 'none'"
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['Referrer-Policy'] = 'same-origin'
    return response
```

F-008: Unpinned and Inconsistent Dependency Manifests

Severity	Medium	CVSS	5.9 CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:H/A:L
	CWE-1104: Use of Unmaintained Third Party Components; CWE-494: Download of Code Without Integrity Check		OWASP Top 10:2025 A06 Vulnerable and Outdated Components
ASVS	ASVS V14 Configuration	Location	pyproject.toml:7-14; backend/pyproject.toml:15-21; backend/requirements.txt:1-6

Description

The repository contains multiple dependency manifests with broad version ranges and a requirements file without pinned versions or hashes.

Technical Details

Root pyproject and backend pyproject specify overlapping but different lower bounds. requirements.txt lists package names only. This can cause inconsistent installs and reduces reproducibility of security fixes.

Evidence

Root dependencies include fastapi>=0.136.3 and bcrypt>=5.0.0. Backend dependencies include fastapi>=0.109.0 and bcrypt>=4.0.0. requirements.txt contains fastapi, uvicorn, python-multipart, starlette, itsdangerous, Bcrypt without versions.

Impact

Builds may resolve to unexpected versions, including vulnerable or incompatible packages, and reviewers cannot easily prove which dependency set is authoritative.

Exploitation Scenario

A future deployment installs from requirements.txt and receives a vulnerable transitive package version because no lockfile/hash policy constrains resolution.

Remediation

Choose one authoritative manifest, maintain a lockfile, pin production versions, enable dependency scanning, and update through reviewed upgrade PRs.

Secure Code Example

```
# Example requirements style for deployment artifacts
fastapi==0.136.3
uvicorn==0.48.0
python-multipart==0.0.30
itsdangerous==2.2.0
bcrypt==5.0.0
```

F-009: Unresolved Merge Conflict Markers in Dashboard Template

Severity	Medium	CVSS	4.3 CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:N/I:L/A:L
	CWE-489: Active Debug Code; CWE-451: User Interface Misrepresentation of Critical Information		OWASP Top 10:2025 A05 Security Misconfiguration
ASVS	ASVS V14 Configuration	Location	frontend/templates/dashboard.html:25-28

Description

The shipped dashboard template contains Git conflict markers and contradictory version messaging.

Technical Details

Conflict markers are rendered as page text around a version announcement. The same page claims SQL injection and MD5 have been fixed while still listing them as vulnerabilities to discover, and repository docs also describe older behavior.

Evidence

dashboard.html contains <<<<<<< HEAD, =====, and >>>>>>> 88f6998 at lines 25-28. The modal says bcrypt and SQL injection fixes were added, while the dashboard vulnerability grid still instructs students to discover SQLi and weak password storage.

Impact

This does not directly compromise data, but it creates integrity and training-risk issues: students and auditors may test stale vulnerabilities or misunderstand the intended lab state.

Exploitation Scenario

A learner follows the dashboard/README instructions and spends time attempting attacks that current code no longer implements, missing the actual remaining issues.

Remediation

Resolve conflict markers, align dashboard copy with the current branch's intended vulnerability set, and add CI checks that fail on conflict marker patterns.

Secure Code Example

```
# CI check example
rg '^(<<<<<<<|=====|>>>>>>>)' --glob '!*.lock' && exit 1 || exit 0
```

F-010: No Security Test, CI, or Vulnerability Response Evidence

Severity	Low	CVSS	2.7 CVSS:3.1/AV:L/AC:H/PR:H/UI:N/S:U/C:N/I:L/A:N
	CWE-1059: Incomplete Documentation of Program Execution; CWE-1104: Use of Unmaintained Third Party Components		OWASP Top 10:2025 A05 Security Misconfiguration
ASVS	ASVS V14 Configuration; secure SDLC supporting controls	Location	Repository root inventory; backend/pyproject.toml:23-26

Description

The repository declares pytest as a dev dependency but contains no tests, CI workflows, SAST/SCA configuration, or vulnerability response documentation.

Technical Details

No test files, GitHub Actions/GitLab CI, pre-commit configuration, dependency scanning, or security policy files were identified in the repository inventory. This weakens change control, especially for an educational app whose intended vulnerable state changes over time.

Evidence

File inventory includes source, docs, manifests, assets, and database; no tests or CI files were present. backend/pyproject.toml lists pytest>=7.0.0 only as an optional dev dependency.

Impact

Security regressions, stale documentation, and accidental production-unsafe changes are more likely to pass unnoticed.

Exploitation Scenario

A future patch reintroduces an injection flaw or ships a conflict marker; without automated checks, the issue is not detected before students or users run it.

Remediation

Add focused tests for intended-vs-fixed vulnerabilities, CI conflict-marker scanning, dependency audit, linting, and a SECURITY.md that states scope, safe-use boundaries, and disclosure handling.

Secure Code Example

```
# pytest example
def test_download_db_requires_auth(client):
    response = client.get('/download/db')
    assert response.status_code in {401, 403, 404}
```

Risk Matrix

Risk	Likelihood	Impact	Priority
Direct compromise of confidentiality/integrity in any reachable deployment.	—	—	—

Risk	Likelihood	Impact	Priority
Abuse facilitation, browser defense gaps, supply-chain uncertainty, and release integrity concerns.	—	—	—
Process maturity gap that increases likelihood of future defects.	—	—	—

Remediation Roadmap

Timeline	Action	Owner	Related Findings
—	—	Engineering	—
—	—	Engineering	—
—	—	Engineering	—
—	—	Engineering	—

NIST SSDF Recommendations

Practice	Recommendation	Evidence / Rationale
PO: Prepare the Organization	Document that this application is lab-only, define acceptable deployment boundaries, and maintain a current vulnerability inventory for instructors and students.	—
PS: Protect the Software	Protect secrets outside source, remove sensitive database files from repositories, and use locked dependency artifacts with review controls.	—
PW: Produce Well-Secured Software	Adopt safe framework defaults: parameterized queries, autoescaped templates, CSRF tokens, secure headers, input validation, and abuse throttling. Add tests that confirm both intentional vulnerable behavior and hardened behavior by branch/release.	—
RV: Respond to Vulnerabilities	Add SECURITY.md, dependency/SAST scanning, conflict-marker detection, and a documented process for reconciling stale docs when vulnerabilities are added, removed, or patched.	—

Final Security Rating

High Risk

Started at 100 and subtracted for confirmed current findings: 4 High findings (48), 5 Medium findings (30), and 1 Low finding (2), yielding 20/100. SQL injection and MD5 were not counted because the current code uses parameterized queries and bcrypt. The intentional lab context explains the risk but does not reduce the technical exposure if run on a reachable network.

Appendix: Standards Mapping and Evidence Notes

This report maps observed repository evidence to OWASP ASVS, OWASP Top 10, CWE, CVSS v3.1, and NIST SSDF. Findings are based on available source code and configuration at the assessed version/commit. Secrets,

when identified, should be rotated even when redacted in this report.